SP-104 Green

Real-Time Object Detection for Traffic Cameras

Final Report Draft

4850 Senior Project | Spring 2026

Professor Perry

Zachary Gray, Bregan Frank, Jeyden Johnson, David Obasi

5/04/2026

GitHub Repository:

https://github.com/grayobasijohnsonfrank-sp104/grayobasijohnsonfrank-sp104

https://grayobasijohnsonfrank-sp104.github.io/grayobasijohnsonfrank-sp104/

STATS AND STATUS	
Lines of Code (LOC)	600+
Components/Tools	Pycharm(IDE), github pages(frontend/hosting), hugging face(build and deployment), docker (container holds build and deployment process), python 3, ultralytics yolov5, etc.
Estimated Hours	401
Estimated Actuals	500
Status	Complete

Table of Contents

Table of Contents 1
1. Introduction 1
  1.1 Project Overview 1
  1.2 Goals 1
  1.3 Project Scope 1
  1.4 Assumptions 1
  1.5 Definitions and Acronyms 1
  1.6 Team Coordination 1
2. Requirements 1
  2.1 Functional Requirements 1
  2.2 Non-Functional Requirements 1
  2.3 External Interface Requirements 1
    2.3.1 User Interface 1
    2.3.2 Hardware Interface 1
    2.3.3 Software Interface 1
    2.3.4 Communication Interface 1
  2.4 Risk Assessment 1
3. Analysis / Design 1
  3.1 Architectural Strategy 1
  3.2 System Architecture 1
  3.3 Tracking Module 1
  3.4 Integration and Interfaces 1
  3.5 Use Cases 1
  3.6 User Stories 1
  3.7 Data Flow 1
  3.8 Constraints 1
4. Development 1
  4.1 Model and Data 1
  4.2 Dataset Structure 1
  4.3 Gathering Data 1
  4.4 Training the Model 1
  4.5 Running Detection 1
  4.6 Database Connection 1
  4.7 Configuration and Deployment 1

5. Test Plan and Report 1
5.1 Test Objectives 1
5.2 Scope of Testing 1
5.3 Test Cases 1
5.4 Test Procedures 1
5.5 Test Environment 1
5.6 Test Data 1
5.7 Software Test Report 1
6. Version Control 1
6.1 Version Control Plan 1
6.2 Repository 1
6.3 Deliverables 1
7. Summary 1
7.1 Conclusion 1
7.2 Future Work 1
Appendix: Usage Guide 1
A.1 Environment Preparation 1
A.2 Dataset Configuration 1
A.3 Model Training 1
A.4 Detection and Validation 1

1. Introduction

1.1 Project Overview

Traffic cameras need the ability to identify objects on the road. With the completion of our Real-Time Object Detection for Traffic Cameras, a common traffic camera will be able to identify vehicles, road signs, and pedestrians from city traffic feeds.

Our program uses datasets of images and trains on this data with CNNs, YOLOv5, and PyTorch. This enables a traffic camera to detect, name, and record the frequency of objects in real time.

1.2 Goals

- Fine-tune a YOLOv5 model to detect traffic-related objects: road signs, pedestrians, and vehicles.
- Support both still images and live/pre-recorded video footage as input.
- Provide high detection accuracy across a variety of footage conditions.

1.3 Project Scope

In Scope:

- Identify road signs, pedestrians, and vehicles.
- Process video footage from real traffic cameras.
- Work with both live and pre-recorded footage.

Out of Scope:

- License plate recognition.
- Non-road transportation (bicycles, scooters, skateboards, etc.).
- Signs unrelated to road traffic.

1.4 Assumptions

- The system will identify pedestrians and vehicles using video footage from real traffic cameras.
- The system will operate on common modern operating systems with internet access.
- Sufficient storage and capable hardware (GPU) are available.
- The user is proficient in the terminal/CLI.
- Footage conditions include clear weather and daylight.
- All footage used is legally obtained, open-source, and compliant with local laws.
- The user does not have any cognitive disabilities or impairments due to flashing colors of bounding boxes around the object.

1.5 Definitions and Acronyms

- CV: Computer Vision — enables AI to interpret meaningful information from images and videos.
- YOLO (You Only Look Once): Object detection algorithm.
- YOLOv5: Version 5 of the YOLO model from Ultralytics.
- AI: Artificial Intelligence.
- IDE: Integrated Development Environment.
- Epoch: One full pass through the training dataset.
- Confidence Score: How confident the model is in a detection (derived from TP, TN, FP, FN).
- NMS (Non-Maximum Suppression): Removes redundant overlapping bounding boxes.
- mAP: Mean Average Precision — standard model evaluation metric.
- PyTorch: Open-source machine learning framework.
- OpenCV: Python library for computer vision tasks.
- Occlusion & Crowding: Dense scenes that cause missed detections or merged bounding boxes.
- Standard RGB: Pixel color format with values 0-255 per channel.

1.6 Team Coordination

Meeting Schedule: Mondays and Fridays at 5:00 PM

Communication Tools: Microsoft Teams and GroupMe (online); Kennesaw Campus Library (in person)

Project Name: RT Object Detction
Report Date: 5/3/2026

					Milestone #1				*Milestone #2*				*Milestone #3*				C-Day	
Deliverable	**Tasks**	**Complete%**	**Current Status Memo**	**Assigned To**	**01/27**	**02/03**	**02/10**	**02/17**	**02/24**	**03/03**	**03/10**	**03/17**	**03/24**	**03/31**	**04/07**	**04/14**	**04/21**	**04/28**
Requirements	Meet with stakeholder(s) SH	100%		All Member	20													
	Define requirements	100%		All Member		12												
	Review requirements with SH	100%		All Member			15											
	Get sign off on requirements	100%		All Member			15	15										
Project design	Define tech required *	100%		Breagen, David				15	12									
	Database design	100%		Not applicable														
	X design	100%		Zachary, Bregan						10	10	12						
	Y design	100%		Jeyden,David						12	12	12						
	Develop working prototype	100%		Zachary, Bregan, Jeyden						10	14	14	5					
	Test prototype	100%		David, Zacher, Bregan							20	10	12					
Development	Review prototype design	100%		Breagen, David								15	10	15				
	Rework requirements	100%		Zachrya, Jeydn									20	20	30	30		
	Document updated design	100%		Jeyden, David											15	15		
	Test product	100%		Zachary, Bregan, Jeyden										10	10	25		
Final report	Presentation preparation	100%		All Members												10	10	5
	Poster preparation	100%		All Members														10
	Final report submission to D2L and project owner	0%		All Members														5
			Total work hours	**512**	**20**	**12**	**30**	**30**	**12**	**32**	**56**	**63**	**47**	**45**	**55**	**80**	**10**	**20**

* formally define how you will develop this project including source code management

Legend

Planned	
Delayed	
Number	Work: man hours

2. Requirements

2.1 Functional Requirements

- A computer or laptop capable of running the model.
- A stable internet connection.
- A live camera (webcam) for real-time detection.
- Support for pre-recorded video input.

2.2 Non-Functional Requirements

- Efficient detection across different video resolutions and frame rates.
- Ability to detect different road sign types.

2.3 External Interface Requirements

2.3.1 User Interface

Buttons for uploading footage and activating the live camera feed.

2.3.2 Hardware Interface

Webcam; GPU with at least 6 GB of VRAM recommended.

2.3.3 Software Interface

Python 3.8> (primary language); PyTorch; YOLOv5.

2.4 Risk Assessment

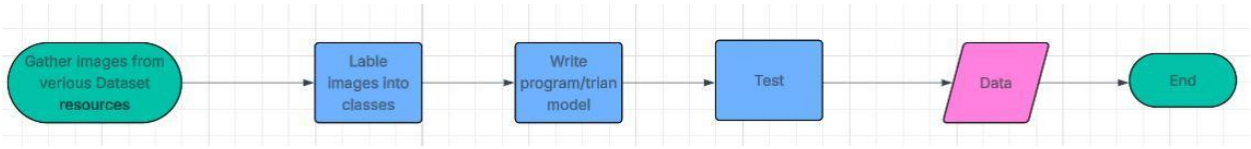
- Misidentification of objects could create hazardous conditions on motorways.
- Source code could potentially be repurposed for malicious use cases.

3. Analysis / Design

3.1 Architectural Strategy

The system is built in Python 3, the standard language for modern object detection development. YOLOv5 was selected due to its strong community support and suitability for real-time traffic object detection. PyTorch serves as the underlying ML framework. Datasets were sourced from COCO and Roboflow (public, non-proprietary).

3.2 System Architecture



Components:

- Ingestion: OpenCV (cv2) for live webcam or pre-recorded video. CLI argument 0 defaults to webcam.
- Detection: YOLOv5 returns bounding boxes, class IDs, and confidence scores. Preprocessing includes resizing, padding, and normalization. Postprocessing includes NMS for bounding box refinement.
- Output: Annotated video stream with labeled bounding boxes and object counts.

3.3 Tracking Module

Output format includes bounding boxes, class IDs, confidence scores, and track IDs.

3.4 Use Cases

Primary: Identify vehicles, pedestrians, and road signs in real-time or from footage.

Potential secondary use: Detecting road violations (unintended).

3.5 User Stories

As a user, I want to:

- Upload video footage or use a live camera to run the model.
- Click a button to start detection.
- See objects outlined with colored bounding boxes.
- See a label on each box describing the detected object.
- See a counter tracking how many of each object type have been detected in a session.

3.6 Data Flow

Collect tailored data → Segment into training/validation sets → Train model with PyTorch and YOLOv5 → Test on held-out data → Record metrics (mAP, precision, recall).

3.7 Constraints

- Hardware: Must have a computer (laptop or desktop) with a working display.
- Connectivity: Stable internet connection required.
- Distance: Detection accuracy depends on how close objects are to the camera.
- Weather: Performance degrades in snow, rain, or low-light conditions.
- Occlusion: Dense scenes may cause merged or missed detections.
- Latency: Real-time video requires processing under 100ms per frame.
- Storage: Model is approximately 200 MB; additional storage needed for footage.
- Color: Input must be in standard RGB format (0-255).
- Resources: NVIDIA GPU (e.g., RTX 3070 Ti) strongly recommended for training.

4. Development

4.1 Model and Data

YOLOv5 is used in a transfer learning setup: the model arrives pre-trained on large-scale data, and its weights are refined using our custom traffic dataset. The final dataset consisted of approximately 1,200 training images across six classes (‘human’, ‘car’, ‘truck’, ‘regulatory’, ‘stop’, ‘warning’), sourced from COCO 2017 and a Roboflow US road signs dataset. Training was performed locally using an NVIDIA RTX 3050 Ti GPU with CUDA 12.1, achieving a final mAP@0.5 of approximately 0.85.

4.2 Dataset Structure

The dataset was packaged as data.zip, containing:

- train/images/ and train/labels/ — training images and corresponding label files.
- valid/images/ and valid/labels/ — validation images and labels.
- classes.txt — lists all class names.
- data.yaml — configuration file mapping paths and class names for YOLOv5.

4.3 Gathering Data

Images were sourced from two public datasets: COCO 2017 (for human, car, and truck classes) and a US Road Signs dataset from Roboflow (for regulatory, stop, and warning sign classes). COCO images were downloaded selectively using FiftyOne, pulling only the 500 most relevant training samples containing the target classes rather than the full 330,000-image dataset. The Roboflow signs dataset provided 664 labeled images. After downloading, class IDs from each dataset were remapped using a custom Python script to align with our unified six-class schema before merging into a single training directory.

4.4 Training the Model

After configuring the yaml file, training was executed with the following command:

```
python train.py --img 640 --batch 16 --epochs 100 --data data.yaml --weights
yolov5s.pt --device 0
```

Key flags: --img sets image size; --batch controls images per training step; --epochs sets total iterations. The best-performing weights (best.pt) are saved automatically.

4.5 Running Detection

After training, detection is run with:

```
python detect.py --weights runs/train/<run>/weights/best.pt --img 640 --conf
0.25 --source "path/to/input"
```

The --conf flag sets the minimum confidence threshold (0.25 = 25%). Below this, detections are not reported.

4.6 Database Connection

Not applicable for this project.

4.7 Configuration and Deployment

The system uses a Miniconda virtual environment (trafficv5) for local dependency management. Configuration is handled via YAML files and CLI arguments. For web deployment, the trained model is served through a FastAPI backend hosted on Hugging Face Spaces inside a Docker container. The frontend is hosted on GitHub Pages and communicates with the backend via HTTP requests.

5. Test Plan and Report

5.1 Test Objectives

- Verify that the model can detect each of the six target object classes.
- Confirm detection works across all three input sources: still images, pre-recorded video, and live webcam.
- Ensure bounding boxes and class labels are rendered correctly.

5.2 Scope of Testing

Testing covers the six trained classes (Human, Car, Truck, Regulatory Sign, Warning Sign, Stop Sign) across three input types (image, video, webcam).

5.3 Test Cases

- Detects all classes from a still image.
- Detect each individual class from a still image.
- Detects all classes from pre-recorded video.
- Detects all classes from live webcam feed.

5.4 Test Procedures

- Run detect.py with the trained best.pt weights.
- Supply image, video, or webcam index as --source.
- Observe bounding boxes, class labels, and confidence scores in the output.

5.5 Test Environment

- OS: Windows 11
- GPU: NVIDIA RTX 3050 Ti
- Environment: Conda virtual environment (trafficv5)
- IDE: PyCharm

5.6 Test Data

- Still images containing any of the 6 target classes.
- Pre-recorded video footage containing any of the 6 target classes.
- Live webcam with real-time footage of any target class within clear view.

5.7 Software Test Report

Requirements/Test Case	Pass	Fail	Severity
Detect all classes (image)	Pass	No	Med
Detect Human (image)	Pass	No	Low
Detect Car (image)	Pass	No	Low
Detect Truck (image)	Pass	No	Med
Detect Regulatory Sign (image)	Pass	No	Low
Detect Warning Sign (image)	Pass	No	Med
Detect Stop Sign (image)	Pass	No	Low
Detect all classes (video)	Pass	No	High
Detect Human (video)	Pass	No	Low
Detect Car (video)	Pass	No	Low
Detect Truck (video)	Pass	No	Med
Detect Regulatory Sign (video)	Pass	No	Low
Detect Warning Sign (video)	Pass	No	Med
Detect Stop Sign (video)	Pass	No	Low
Detect all classes (webcam)	Pass	No	Med
Detect Human (webcam)	Pass	No	Low
Detect Car (webcam)	Pass	No	Low
Detect Truck (webcam)	Pass	No	Med
Detect Regulatory Sign (webcam)	Pass	No	Low
Detect Warning Sign (webcam)	Pass	No	Med
Detect Stop Sign (webcam)	Pass	No	Low

6. Version Control

6.1 Version Control Plan

The team used GitHub for version control. Each team member was added as a collaborator on the group repository. Work was isolated to individual branches following industry-standard practices. Each commit includes a detailed message describing the changes made. All final deliverables will be publicly available in the repository.

6.2 Repository

GitHub: https://github.com/grayobasijohnsonfrank-sp104/grayobasijohnsonfrank-sp104

Project Website: https://grayobasijohnsonfrank-sp104.github.io/grayobasijohnsonfrank-sp104/

Hugging face space: https://huggingface.co/sp104greentraff

6.3 Deliverables

- Real-time inference demo
- Trained YOLOv5 model (best.pt)
- mAP metrics and test report
- Source code (available on GitHub)

7. Summary

7.1 Conclusion

The YOLOv5 object detection model was successfully trained and deployed as a functional local and web application. The model detects objects across six road-related classes: Human, Car, Truck, Regulatory Sign, Warning Sign, and Stop Sign, achieving a mAP@0.5 of approximately 0.85. Detection is accessible through a public-facing website hosted on GitHub Pages, where users can upload images and receive annotated results with bounding boxes and confidence scores returned from the Hugging Face backend.

Some classes exhibited higher false positive rates, likely due to class imbalance in the training dataset. The initial dataset also suffered from class ID mismatches across sources, which required a remapping script to correct before retraining. Once the dataset pipeline was corrected and training was run on GPU, overall model performance improved significantly.

7.2 Future Work

- Expand and balance the training dataset with more images per class to improve accuracy.
- Extend web interface to support video upload and live webcam detection via WebSocket connections.
- Add analytics features such as heatmaps, dwell time, and route analysis.

8. Current State

We successfully deployed a functional trained yolov5 model that works locally as well as a web application that performs object detection on user-uploaded images. Our initial scope included webcam support and live video streaming; however, due to the excess overhead of processing video frames on a remote server and the complexity of managing WebSocket connections at scale, we prioritized delivering a stable and reliable image detection pipeline within our project timeframe.

To enable image detection on our website, we implemented a full-stack architecture connecting our GitHub Pages frontend to a backend inference server running our trained YOLOv5 model.

We built the backend using FastAPI, a Python web framework that allowed us to expose our YOLOv5 model as an HTTP API endpoint. This was necessary because GitHub Pages only supports static files — it can't execute Python or run server-side code, so the model inference had to happen on a separate server.

Our first deployment attempt used Render, a cloud hosting platform we connected to our GitHub repository. However, Render's free tier only provides 512MB of RAM, which was insufficient to load our PyTorch and YOLOv5 dependencies. The server consistently ran out of memory during startup, making it unusable for our purposes.

We then switched to Hugging Face Spaces, which provided 16GB of RAM on its free tier — well above the approximately 600MB our model requires to load. Hugging Face Spaces also supports Docker, which gave us full control over the deployment environment. We configured a Docker container that installs all required dependencies from our requirements file, loads the YOLOv5 model weights from our trained best.pt file using the Ultralytics library, and starts our FastAPI server on port 7860.

On the frontend, our GitHub Pages site uses a fetch call in index.html to communicate with the Hugging Face backend. When a user uploads an image, the browser sends an HTTP POST request to our /detect/image endpoint. The FastAPI server receives the image, runs it through the YOLOv5 model, renders bounding boxes and confidence scores onto the image, and returns the annotated result. The frontend then displays the returned image directly on the page.

Appendix: Usage Guide

A.1 Environment Preparation

Hardware: Ensure the machine has an NVIDIA GPU (e.g., RTX 3050 Ti). Install Miniconda from https://docs.conda.io/en/latest/miniconda.html and add it to your system PATH.

```
git clone https://github.com/grayobasijohnsonfrank-sp104/grayobasijohnsonfrank-sp104
```

```
cd yolov5
```

```
conda create --name trafficv5 python=3.11 conda activate trafficv5 cd yolov5 pip install -r requirements.txt pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
```

A.2 Dataset Configuration

Upload and unzip your dataset into the local environment. Create or edit a .yaml file:

```
train: path/to/train/images
```

```
val: path/to/valid/images
```

```
nc: 6
```

```
names: ['human', 'car', 'truck', 'regulatory', 'stop', 'warning']
```

A.3 Model Training

Run the training script using the GPU (do NOT use CPU for training):

```
python train.py --img 640 --batch 16 --epochs 100 --data data.yaml --weights yolov5s.pt --name my_run --device 0
```

Trained weights are saved to runs/train/<run_name>/weights/best.pt.

A.4 Detection and Validation

```
python detect.py --weights runs/train/<run>/weights/best.pt --img 640 --conf 0.1 --save-conf --source "C:/path/to/image.jpg"
```

Replace the --source path with your image, video file, or 0 for webcam.